



UNIVERSITAS
BRAWIJAYA



brone
BRAWIJAYA ONLINE LEARNING



FILKOM
FAKULTAS ILMU KOMPUTER

Rekayasa Perangkat Lunak S2

12 Pengujian Perangkat Lunak

Dosen: Dr. Achmad Arwan, S.Kom., M.Kom.

Magister Ilmu Komputer

FILKOM

2026



filkom.ub.ac.id

Learning Objectives

Setelah mempelajari modul ini, mahasiswa mampu:

1. Memahami V-Model dan fase pengujian pada SDLC
2. Merancang unit test untuk class dan API microservices
3. Merancang integration test dengan pendekatan top-down, bottom-up, sandwich
4. Merancang system test dengan **black box** (EP, BVA)
5. Merancang validation & acceptance test
6. Mengukur usability dengan **SUS (System Usability Scale)**
7. Menghubungkan monolitik dan microservices dalam alur end-to-end

Studi Kasus: E-Commerce Order System

Sistem E-Commerce adalah aplikasi yang memungkinkan pelanggan memesan produk, sistem mengelola inventaris, validasi stok, dan menghitung harga total pesanan secara otomatis.

Entitas Utama:

Product

- id
- name
- price
- stock

Order

- id
- productId
- qty
- totalPrice

Masalah & Risiko (Problem Context)

Hal-hal yang mungkin terjadi jika sistem tidak diuji dengan baik:

- **Validasi gagal:** qty negatif atau nol diterima sistem
- **Stok tidak cukup:** order diproses meski stok habis
- **Perhitungan salah:** total price tidak sesuai formula ($\text{price} \times \text{qty}$)
- **Integrasi gagal:** order service tidak bisa akses inventory database
- **Business rule dilanggar:** order melebihi limit maksimal
- **Data inkonsisten:** stok terurasi ganda pada order bersamaan

Fokus pengujian: cegah semua masalah ini dengan strategi test yang terstruktur.

Dua Bentuk Sistem Order

Monolitik

- UI, service, dan DB satu aplikasi
- `OrderService` memanggil repository langsung
- Bug sering muncul di logic internal

Slide berikut selalu memakai studi kasus yang sama, tetapi kita lihat dari dua bentuk arsitektur ini.

Microservices

- Order dan Inventory dipisah service
- Komunikasi lewat HTTP/API
- Bug sering muncul di contract dan network

V-Model Software Testing

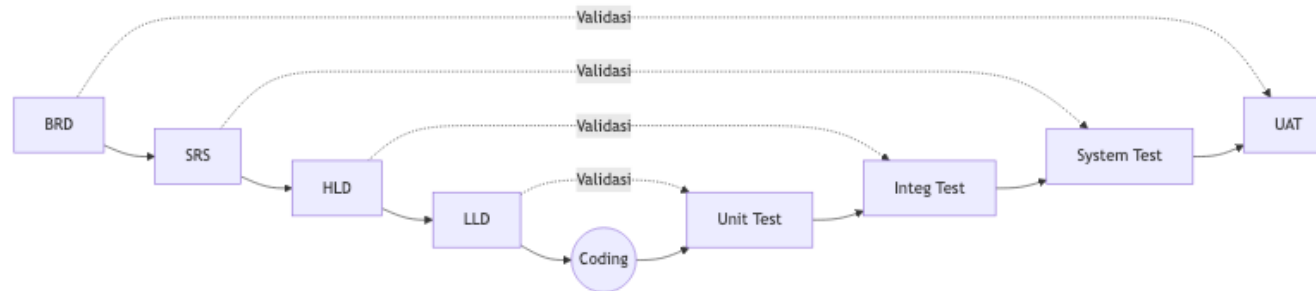
V-Model adalah pendekatan SDLC di mana setiap fase pengembangan dipasangkan dengan fase pengujian setara, sehingga strategi verifikasi disiapkan sejak awal.

Kiri = Development, Kanan = Testing, Garis Putus = Validasi

Setiap dokumen memiliki pasangan tes:

- **Business Requirements Document (BRD) → Acceptance Testing (UAT)**
- **Design (SRS) → System Testing**
- **Architecture (HLD) → Integration Testing**
- **Module (LLD) → Unit Testing**

V-Model — Diagram Alur



Development turun dari BRD ke LLD, bertemu di **Coding** (bawah), lalu Testing naik dari Unit Test ke UAT. Garis putus: setiap fase development dipasangkan validasi dengan fase testing yang sesuai.

Mapping Development vs Testing

Development Phase	Testing Phase	Contoh pada E-Commerce
Requirement (BRD)	UAT / Acceptance	User bisa order produk, stok otomatis berkurang
Design (SRS)	System Testing	Order qty harus ≥ 1 , $\text{price} = \text{qty} \times \text{unit_price}$
Architecture (HLD)	Integration Testing	OrderService terhubung dengan InventoryService
Module (LLD)	Unit Testing	Function <code>validateOrder()</code> menolak $\text{qty} \leq 0$

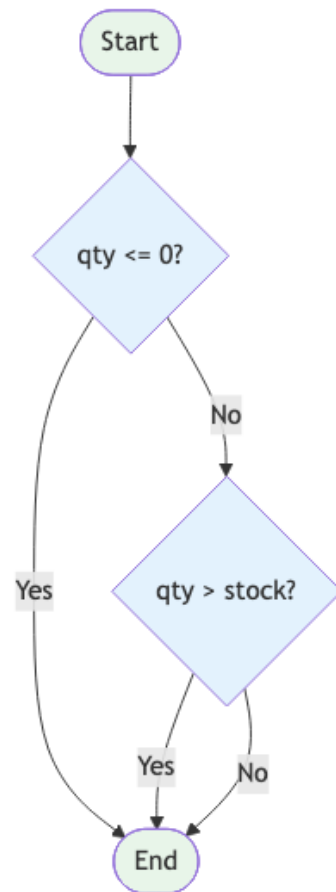
Unit Testing — Logic Internal Monolitik

White Box Testing memeriksa logika internal kode berdasarkan alur kontrol (Control Flow Graph). Setiap jalur eksekusi yang mungkin (basis path) harus tercakup oleh minimal satu test case.

Target pada E-Commerce: method `validateOrder(int qty, int stock)` dengan aturan:

- `qty` harus > 0
- `qty` tidak boleh melebihi `stock`
- Jika keduanya valid, return `true`

Unit Testing — Control Flow Graph (CFG)



Struktur CFG dengan 3 Region:

- **S** = Entry node
- **N1, N2** = Predicate nodes (decision point)
- **Region 1:** S → N1 → Exit (qty <= 0 → false)
- **Region 2:** S → N1 → N2 → Exit (qty > 0 && qty > stock → false)
- **Region 3:** S → N1 → N2 → Exit (qty > 0 && qty <= stock → true)

Return yang sama disatukan ke satu exit node. Basis path = jumlah region = 3.

Unit Testing — Logic Internal Monolitik: Kode

```
public class OrderValidator {  
    public static boolean validateOrder(int qty, int stock) {  
        if (qty <= 0) {           // Node 1  
            return false;        // Node 2  
        }  
        if (qty > stock) {        // Node 3  
            return false;        // Node 4  
        }  
        return true;             // Node 5  
    }  
}
```

Kode ini memiliki **3 basis path** yang perlu diuji — setiap return harus dieksekusi.

Basis Path (Independent Path)

Path ID	Jalur	Kondisi	Deskripsi
P1	$S \rightarrow N1 \rightarrow E$	$qty \leq 0$	Qty tidak valid
P2	$S \rightarrow N1 \rightarrow N2 \rightarrow E$	$qty > 0 \ \& \ qty > stock$	Melebihi stok
P3	$S \rightarrow N1 \rightarrow N2 \rightarrow E$	$qty > 0 \ \& \ qty \leq stock$	Order valid

Unit Testing — Logic Internal Monolitik: Test Case

Test ID	Path	qty	stock	Expected
TC-01	P1	0	10	false
TC-02	P2	11	10	false
TC-03	P3	5	10	true

Unit Testing — Logic Internal Monolitik: Test Java

```
public class OrderValidatorTest {  
    // TC-01: Path P1 - Qty tidak valid (qty <= 0)  
    @Test void testPath1_QtyInvalid() {  
        assertFalse(OrderValidator.validateOrder(0, 10));  
    }  
  
    // TC-02: Path P2 - Melebihi stok (qty > stock)  
    @Test void testPath2_OverStock() {  
        assertFalse(OrderValidator.validateOrder(11, 10));  
    }  
  
    // TC-03: Path P3 - Order valid  
    @Test void testPath3_ValidOrder() {  
        assertTrue(OrderValidator.validateOrder(5, 10));  
    }  
}
```

Unit Testing — API Contract Microservices

Pada arsitektur microservices, unit testing tidak berhenti di method internal. Anda juga perlu menguji kontrak API pada level controller agar parsing JSON, validasi request, dan status HTTP tetap benar tanpa menjalankan service lain.

Fokus unit test API Order Microservice:

- endpoint `POST /api/orders`
- validasi `productId` dan `qty`
- response body JSON
- status `201` , `400` , atau `409`

Unit Testing — API Contract: Spec

Request

```
{  
  "productId": "P-001",  
  "qty": 3  
}
```

- productId wajib ada
- qty harus ≥ 1

Response Sukses

```
{  
  "orderId": "ORD-101",  
  "status": "PENDING",  
  "totalPrice": 150000  
}
```

- 201 jika order valid
- 400 jika input salah
- 409 jika stok tidak cukup

Unit Testing — API Contract: DTO Request

DTO membantu Anda memvalidasi input sejak request masuk, sebelum logic bisnis berjalan:

```
class OrderRequest {  
    @NotBlank  
    private String productId;  
  
    @Min(1)  
    private int qty;  
}
```

Dengan DTO ini, test HTTP 400 bisa fokus pada request validation, bukan pada service logic.

Unit Testing — API Contract: Controller

Controller menerima request JSON, memanggil service, lalu mengembalikan status HTTP yang sesuai:

```
@RestController
@RequestMapping("/api/orders")
class OrderController {
    @PostMapping
    ResponseEntity<OrderResponse> create(
        @RequestBody OrderRequest request) {
        OrderResponse result = orderService.create(request);
        return ResponseEntity.status(201).body(result);
    }
}
```

Pada unit test, Anda mengisolasi controller ini dari database dan service lain.

Unit Testing — API Contract: Error Handler

HTTP 409 biasanya tidak lahir langsung dari controller, tetapi dari exception handler yang memetakan exception bisnis menjadi response API:

```
@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler({StockNotEnoughException.class})
    ResponseEntity<Map<String, String>> handleStock() {
        return ResponseEntity.status(409)
            .body(Map.of("error", "Stock not enough"));
    }
}
```

Dengan handler ini, Anda sekarang paham dari mana status 409 Conflict berasal saat stok tidak cukup.

Unit Testing — API Contract: Test Sukses

Uji controller secara terisolasi dengan `@WebMvcTest` dan mock `OrderService`:

```
@WebMvcTest(OrderController.class)
class OrderControllerUnitTest {
    @Autowired MockMvc mockMvc;
    @MockBean OrderService orderService;

    @Test
    void testCreateOrder_Success() throws Exception {
        when(orderService.create(any()))
            .thenReturn(new OrderResponse("ORD-101", "PENDING", 150000));

        mockMvc.perform(post("/api/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"productId\":\"P-001\",\"qty\":3}"))
            .andExpect(status().isCreated());
    }
}
```

Unit Testing — API Contract: Test 409

Gunakan mock service untuk mensimulasikan stok tidak cukup, lalu verifikasi HTTP 409 Conflict:

```
@Test
void testCreateOrder_StockNotEnough() throws Exception {
    when(orderService.create(any()))
        .thenReturn(new StockNotEnoughException("Stock not enough"));

    mockMvc.perform(post("/api/orders")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"productId\":\"P-001\",\"qty\":50}"))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.error")
            .value("Stock not enough"));
}
```

Tes ini membuktikan kontrak API error tetap konsisten tanpa memanggil Inventory Service yang nyata.

Unit Testing — API Contract: Test Case

ID	Endpoint	Skenario	Request	Expected
UT-MS-01	POST /api/orders	Happy path	<code>{"productId":"P-001","qty":3}</code>	201 + orderId
UT-MS-02	POST /api/orders	qty = 0	<code>{"productId":"P-001","qty":0}</code>	400 + error JSON
UT-MS-03	POST /api/orders	qty negatif	<code>{"productId":"P-001","qty":-1}</code>	400 + error JSON
UT-MS-04	POST /api/orders	productId kosong	<code>{"productId":"","qty":3}</code>	400 + error JSON
UT-MS-05	POST /api/orders	stok tidak cukup	valid request	409 + error

Integration Testing

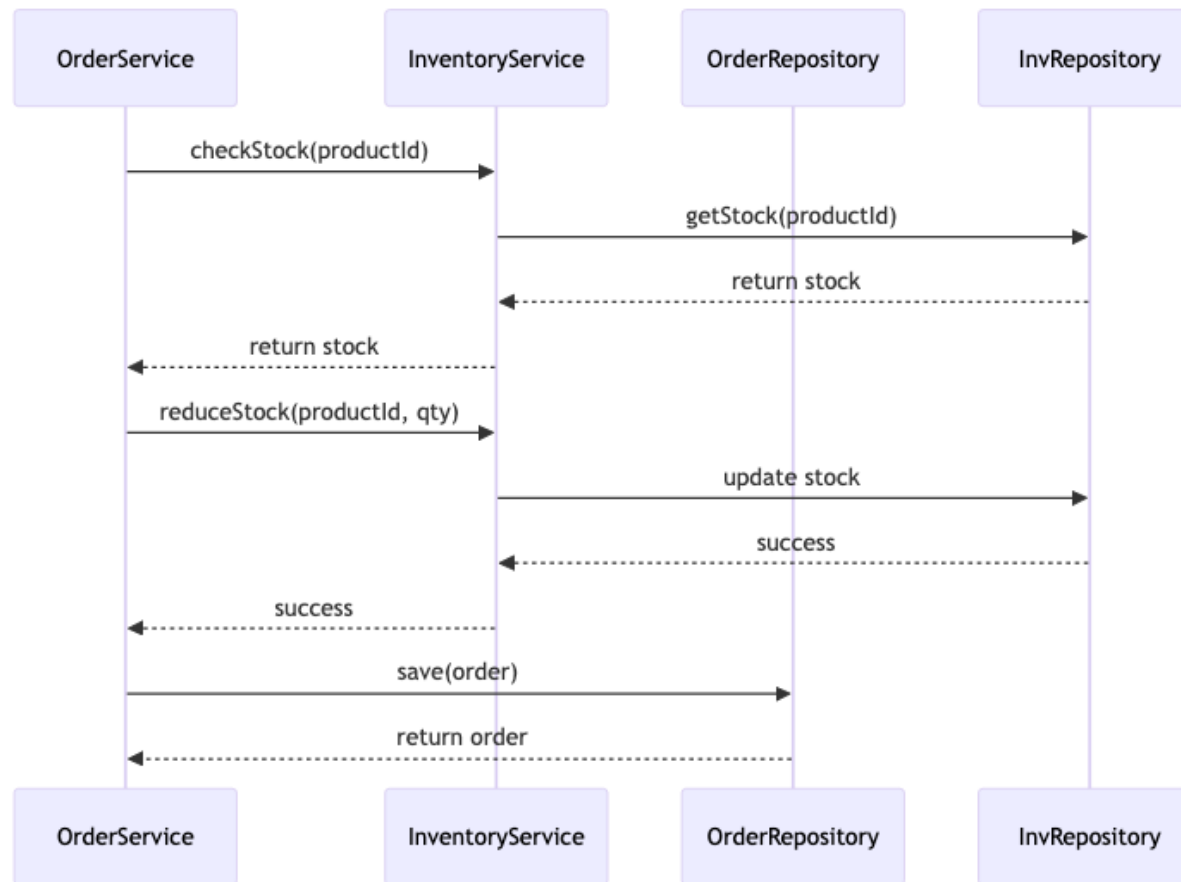
Integration Testing memverifikasi kerja sama antar modul/service. Modul yang belum selesai digantikan oleh Stub (pengganti Callee) atau Driver (pengganti Caller).

Setelah unit test lolos, kita naik satu level: bukan lagi menguji satu class, tetapi hubungan antar komponen pada sistem order.

Tiga strategi integrasi:

- **Top-Down:** mock service bawahan, misalnya `InventoryService`
- **Bottom-Up:** pakai driver untuk menguji repository lebih dulu
- **Sandwich:** service eksternal dimock, database/repository nyata

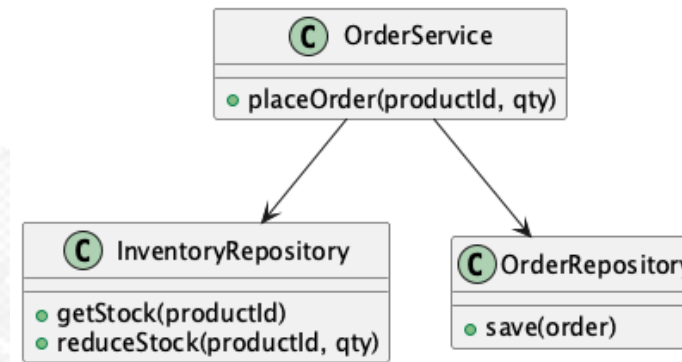
Integration Testing — Diagram Integrasi E-Commerce



Integration Testing — Monolitik: Contoh

Strategi: Top-Down — Uji service utama (`OrderService`) dengan mengganti dependency-nya menggunakan **stub**.

Analogi: Seperti menguji kemampuan manajer proyek tanpa perlu tim asli bekerja — cukup pakai asisten tiruan yang selalu bilang "Oke, sudah selesai!" agar fokus ke logika si manajer, bukan ke pekerjaan timnya.



```

@Test void testPlaceOrder_Monolith() {
    // STUB: Pengganti InventoryRepository nyata
    InventoryRepository invRepo = new FakeInventoryRepository(10);

    // STUB: Pengganti OrderRepository nyata
    OrderRepository orderRepo = new InMemoryOrderRepository();

    // Komponen yang diuji
    OrderService service = new OrderService(invRepo, orderRepo);

    Order result = service.placeOrder("P-001", 2);

    assertEquals(8, invRepo.getStock("P-001"));
    assertEquals("PENDING", result.getStatus());
}

```

Penjelasan: FakeInventoryRepository dan InMemoryOrderRepository adalah **stub** — objek tiruan yang memberikan data tetap tanpa koneksi DB nyata.

Integration Testing — Microservices: Contoh

Strategi: Top-Down — Uji Order Service dengan **stub HTTP** untuk Inventory Service.

```
@Test void testPlaceOrder_Microservice() {  
    // STUB: Mock HTTP response dari Inventory Service  
    stubFor(get(urlEqualTo("/inventory/P-001"))  
        .willReturn(okJson("{\"stock\":10"})));  
  
    stubFor(post(urlEqualTo("/inventory/reduce"))  
        .willReturn(okJson("{\"success\":true"})));  
  
    // Panggil Order Service yang nyata  
    OrderResponse result = orderClient.create("P-001", 2);  
  
    assertEquals("PENDING", result.getStatus());  
    verify(postRequestedFor(urlEqualTo("/inventory/reduce")));  
}
```

Penjelasan: `stubFor()` dari WireMock adalah **stub** — server HTTP tiruan yang memalsukan respons dari Inventory Service tanpa menjalankan service nyata.

Integration Testing — Test Case Monolitik

ID	Strategi	Skenario	Dependency	Expected
IT-M-01	Bottom-Up	Simpan order ke DB	Real repo	Row order tersimpan
IT-M-02	Top-Down	Stok cukup	Fake repo	Status PENDING
IT-M-03	Top-Down	Stok habis	Fake repo	Order ditolak

Integration Testing — Test Case Microservices

ID	Strategi	Skenario	Dependency	Expected
IT-MS-01	Top-Down	Inventory sukses	Mock HTTP	Order sukses
IT-MS-02	Sandwich	Inventory timeout	Mock error + real repo	Error tertangani
IT-MS-03	Sandwich	Order simpan lokal + cek stok remote	Mixed	Data dan API konsisten

System Testing Black Box

System Testing menguji perilaku sistem end-to-end tanpa melihat internal kode. Fokus pada requirement fungsional, batas nilai (boundary), dan kategori input (equivalence).

Teknik Black Box yang dipakai:

- **Equivalence Partitioning (EP):** kelompokkan input ke kategori valid/invalid
- **Boundary Value Analysis (BVA):** uji nilai di perbatasan kategori

Skenario E-Commerce: order qty maksimal 100 unit per transaksi.

System Testing — Monolitik vs Microservices

Monolitik

- UI -> controller -> service -> DB
- Jalur E2E lebih pendek
- Fokus pada output bisnis akhir

Karena itu, test case EP dan BVA berikut dipakai pada dua arsitektur, tetapi titik observasinya berbeda.

Microservices

- UI -> API Gateway -> Order -> Inventory
- Jalur E2E lebih panjang
- Fokus pada output dan contract antar API

System Testing — Equivalence Partitioning (EP)

Equivalence Partitioning (EP) adalah teknik black box testing yang membagi rentang input menjadi kelas-kelas ekuivalen. Semua nilai dalam satu kelas diasumsikan berperilaku sama, sehingga cukup uji satu perwakilan dari setiap kelas.

Analogi: Seperti mencicipi satu sendok dari setiap panci masakan — kalau satu sendok enak, seluruh panci kemungkinan besar enak. Tidak perlu coba semua isi panci.

Untuk order qty dengan batas maksimal 100:

Kategori EP:

- **Valid:** $1 \leq \text{qty} \leq 100$
- **Invalid — terlalu kecil:** $\text{qty} < 1$ (contoh: 0, -5)
- **Invalid — terlalu besar:** $\text{qty} > 100$ (contoh: 101, 500)

Perwakilan EP (satu test case per kategori):

- Valid kelas: $\text{qty} = 50$ (tengah range)
- Invalid kecil: $\text{qty} = 0$ atau -1
- Invalid besar: $\text{qty} = 101$ atau 200

System Testing — EP Test Case Table

ID	Kategori EP	Input qty	Expected Status	Expected Response
ST-EP-01	Valid — rendah	1	201 Created	Order diterima
ST-EP-02	Valid — tengah	50	201 Created	Order diterima
ST-EP-03	Valid — tinggi	100	201 Created	Order diterima
ST-EP-04	Invalid — kecil	0	400 Bad Req	"Qty harus ≥ 1 "
ST-EP-05	Invalid — besar	101	400 Bad Req	"Max order 100 unit"

System Testing — Boundary Value Analysis (BVA)

Boundary Value Analysis (BVA) adalah teknik black box testing yang fokus menguji nilai tepat di batas antara kelas input valid dan invalid. Bug paling sering muncul di titik batas, bukan di tengah range.

Analogi: Seperti menguji apakah pintu otomatis buka saat orang berdiri persis di garis sensor — bukan di tengah ruangan. Titik batas itulah yang paling kritis.

Untuk qty maksimal 100, nilai batas adalah **0, 1, 100, 101**:

BVA Test Points:

- **Lower Bound Valid:** qty = 1 (paling kecil yang diterima)
- **Lower Bound Invalid:** qty = 0 (paling besar yang ditolak)
- **Upper Bound Valid:** qty = 100 (paling besar yang diterima)
- **Upper Bound Invalid:** qty = 101 (paling kecil yang ditolak)

Strategi: selalu uji nilai tepat di garis batas, bukan di tengah kategori.

System Testing — BVA Test Case Table

ID	Tipe Batas	Input qty	Batas Tipe	Expected
ST-BVA-01	Lower Invalid	0	Tepat di batas	Ditolak (400)
ST-BVA-02	Lower Valid	1	Tepat di batas	Diterima (201)
ST-BVA-03	Upper Valid	100	Tepat di batas	Diterima (201)
ST-BVA-04	Upper Invalid	101	Tepat di batas	Ditolak (400)
ST-BVA-05	Just beyond	102	Di luar batas	Ditolak (400)

Validation Testing

Validation Testing memastikan bahwa sistem dibangun sesuai requirement yang sudah ditentukan di SRS (System Requirements Specification). Fokusnya adalah: "Apakah kami membangun sistem yang BENAR?"

Persyaratan pada E-Commerce (dari SRS):

- Order tidak boleh melebihi stok yang tersedia
- Order qty harus positif (≥ 1)
- Total price = qty $\times$ unit_price (tidak boleh salah hitung)
- Status default order adalah "PENDING"

Validation Testing — Monolitik vs Microservices

Monolitik

- Validasi di service/domain object
- Cek state object setelah proses
- Cocok untuk rule dan entity lifecycle

Dengan pola ini, requirement tetap sama, tetapi tempat kita mengamatinya berbeda.

Microservices

- Validasi di API dan service boundary
- Cek JSON response dan status code
- Cocok untuk contract dan konsistensi data

Validation Testing — Test Case Table

ID	Requirement	Skenario	Input	Expected
VAL-01	Qty $\geq$ 1	User input qty = 0	qty=0, stok=10	Ditolak
VAL-02	Qty $\leq$ stok	User order > stok	qty=15, stok=10	Ditolak
VAL-03	Total harus benar	Hitung 5 x Rp100.000	qty=5, price=100k	Total=500k
VAL-04	Status default	Order baru dibuat	new Order()	status="PENDING"
VAL-05	Stok berkurang	Order berhasil	qty=5, stok=10	stok baru=5

User Acceptance Testing

User Acceptance Testing (UAT) adalah pengujian akhir oleh pengguna nyata untuk memastikan sistem memenuhi kebutuhan bisnis. Fokusnya adalah: "Apakah sistem ini sesuai harapan bisnis?" bukan kebutuhan teknis.

Dua tipe UAT:

- **Alpha Testing:** internal, tim developer melakukan task bisnis
- **Beta Testing:** eksternal, pengguna nyata menggunakan aplikasi

Metrik: System Usability Scale (SUS) — kuesioner 10 pertanyaan untuk mengukur usabilitas.

UAT — Fokus Pengujian

Dari sudut pengguna, alur bisnis tetap sama: cari produk, pilih qty, checkout. Yang dinilai adalah apakah sistem memenuhi harapan bisnis, bukan detail teknisnya.

- User menilai keseluruhan pengalaman, bukan arsitektur di balik layar
- Gangguan yang muncul bisa dari bug logic, UI yang membingungkan, atau data yang tidak konsisten
- UAT berhasil jika pengguna bisa menyelesaikan task bisnis tanpa hambatan

UAT — Test Case Task-Based

ID	Skenario Task	User Action	Expected Outcome
UAT-01	Beli produk	Cari produk, input qty=2, checkout	Order sukses, email konfirmasi
UAT-02	Stok habis	Cari produk stok=0, coba order	Sistem tolak, pesan error jelas
UAT-03	Qty invalid	Input qty=-1 atau qty=0	Validasi tolak, user notif
UAT-04	Multi-item	Order item A(qty=2), item B(qty=3)	Kedua item diproses, total benar
UAT-05	Pembayaran	Lengkapi form, klik Bayar	Transaksi sukses, bukti dikirim

SUS (System Usability Scale)

SUS adalah kuesioner standar internasional (10 pertanyaan, skala 1-5) untuk mengukur seberapa usable (mudah digunakan) suatu sistem. Hasil adalah skor 0-100, dengan target ≥ 68 berarti acceptable.

Cara kerja:

- Pengguna menjawab 10 pertanyaan dengan skala 1-5 (1=STS, 5=SS)
- Item ganjil = positif, item genap = negatif
- Konversi: item positif (jawaban-1), item negatif (5-jawaban)
- Skor = (total 10 item) $\times$ 2.5 = nilai 0-100

SUS — 10 Pertanyaan Standar

No	Pertanyaan	Skala
1	Saya akan sering memakai sistem ini	1-5
2	Sistem ini terlalu rumit untuk digunakan	1-5
3	Sistem ini mudah digunakan	1-5
4	Saya memerlukan bantuan dari tim teknis	1-5
5	Fitur-fitur sistem terintegrasi dengan baik	1-5
6	Ada terlalu banyak ketidakkonsistenan dalam sistem	1-5
7	Mayoritas orang akan belajar menggunakan sistem ini dengan cepat	1-5
8	Sistem ini sangat sulit digunakan	1-5
9	Saya merasa sangat percaya diri ketika menggunakan sistem	1-5
10	Saya perlu belajar banyak hal sebelum menggunakan sistem	1-5

SUS — Cara Hitung Skor

Rumus konversi per item:

- Item 1,3,5,7,9 (positif): nilai - 1
- Item 2,4,6,8,10 (negatif): 5 - nilai

Contoh: User menjawab Q1=4, Q2=2, Q3=5, Q4=1 (dst):

Item	Tipe	Jawaban	Skor
Q1	Positif	4	$4-1=3$
Q2	Negatif	2	$5-2=3$
Q3	Positif	5	$5-1=4$
Q4	Negatif	1	$5-1=4$

Skor akhir: $(3+3+4+4+...dst) \times 2.5$

SUS — Interpretasi Skor

Skor Numerik

- 85-100: Excellent
- 71-85: Good
- 52-71: OK
- 41-52: Poor
- 0-41: Not Acceptable

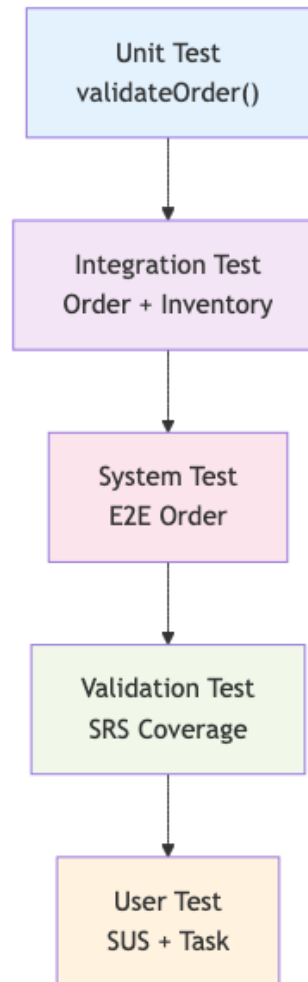
Adjective Rating

- 80-100: Promoters
- 51-80: Passable
- ≤ 50 : Detractors

Target: Skor ≥ 68 untuk lulus UAT usability.

End-to-End Flow Testing

Semua fase pengujian saling terhubung dalam satu alur:



End-to-End

1. **Unit:** mulai dari rule qty dan stock
2. **Integration:** pastikan order bicara benar ke dependency
3. **System:** uji skenario order dari luar sistem
4. **Validation:** cocokkan hasil dengan requirement SRS
5. **User:** buktikan task bisnis benar-benar selesai

Pada monolitik, alurnya lebih pendek. Pada microservices, alurnya melewati lebih banyak boundary, tetapi ceritanya tetap sama: user ingin order berhasil dengan data yang benar.

Mapping Phase ke Fokus & Contoh

Phase	Fokus	Contoh pada E-Commerce
Unit	Kode / contract lokal	Monolitik: <code>validateOrder()</code> ; Micro: <code>POST /api/orders</code>
Integration	Dependency saling bicara	Monolitik: service -> repo; Micro: Order -> Inventory API
System	End-to-end (black box)	Monolitik: UI -> DB; Micro: UI -> Gateway -> services
Validation	Sesuai SRS requirement	Monolitik: state object; Micro: status code + JSON
User	Bisnis dan usability	Checkout sukses pada dua arsitektur

Tools & Library Pengujian

Unit Testing

- JUnit 5 — framework test Java
- Mockito — mock object dependency
- Jacoco — code coverage report

Integration & System

- WireMock — HTTP stub untuk API
- RestAssured — E2E API testing
- Selenium — UI automation testing

Kesimpulan

1. **V-Model dulu:** pahami requirement sebelum bikin tes
2. **CFG untuk white box:** gambar control flow untuk setiap path
3. **Isolasi unit test:** jangan sentuh DB nyata, gunakan mock
4. **Stub untuk integrasi:** gunakan WireMock jika service belum siap
5. **BVA untuk batas:** uji nilai 0, 1, 100, 101 — bukan tengah range
6. **Tabel test case:** dokumentasikan semua scenario per fase
7. **SUS untuk UAT:** jangan lupa mengukur usability dengan kuesioner
8. **Dua arsitektur, satu cerita:** jaga contoh monolitik dan microservices tetap konsisten